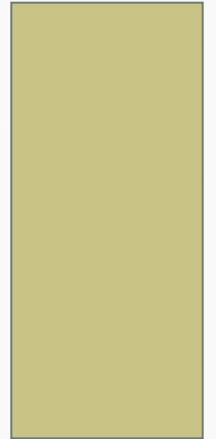


EC-200 DATA STRUCTURES

LECTURE 17



HOW CAN WE MEASURE EFFICIENCY

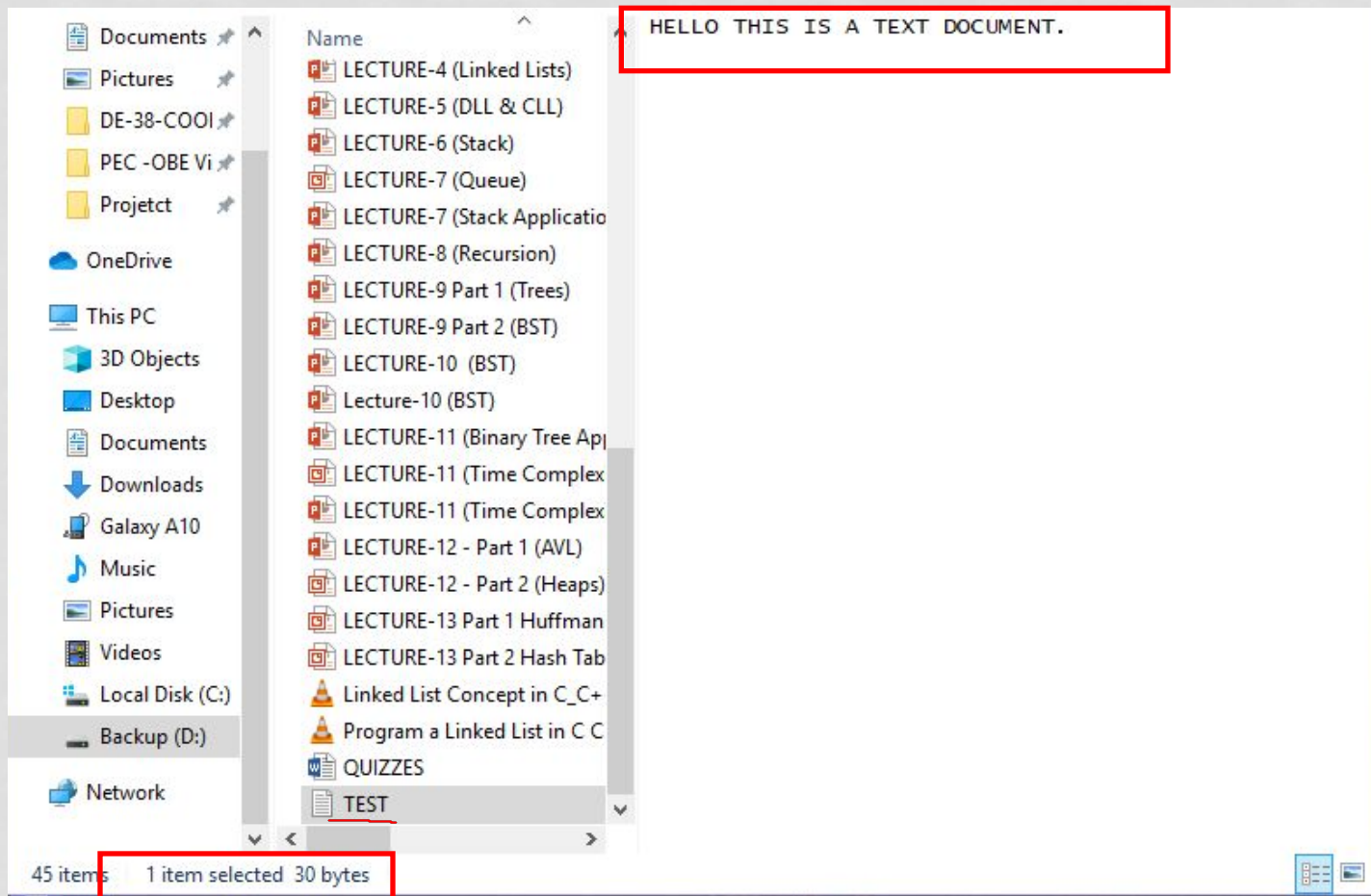
- Time
- Space

WHY WE NEED DATA COMPRESSION

- To reduce space complexity
- To enhance the data transmission speed

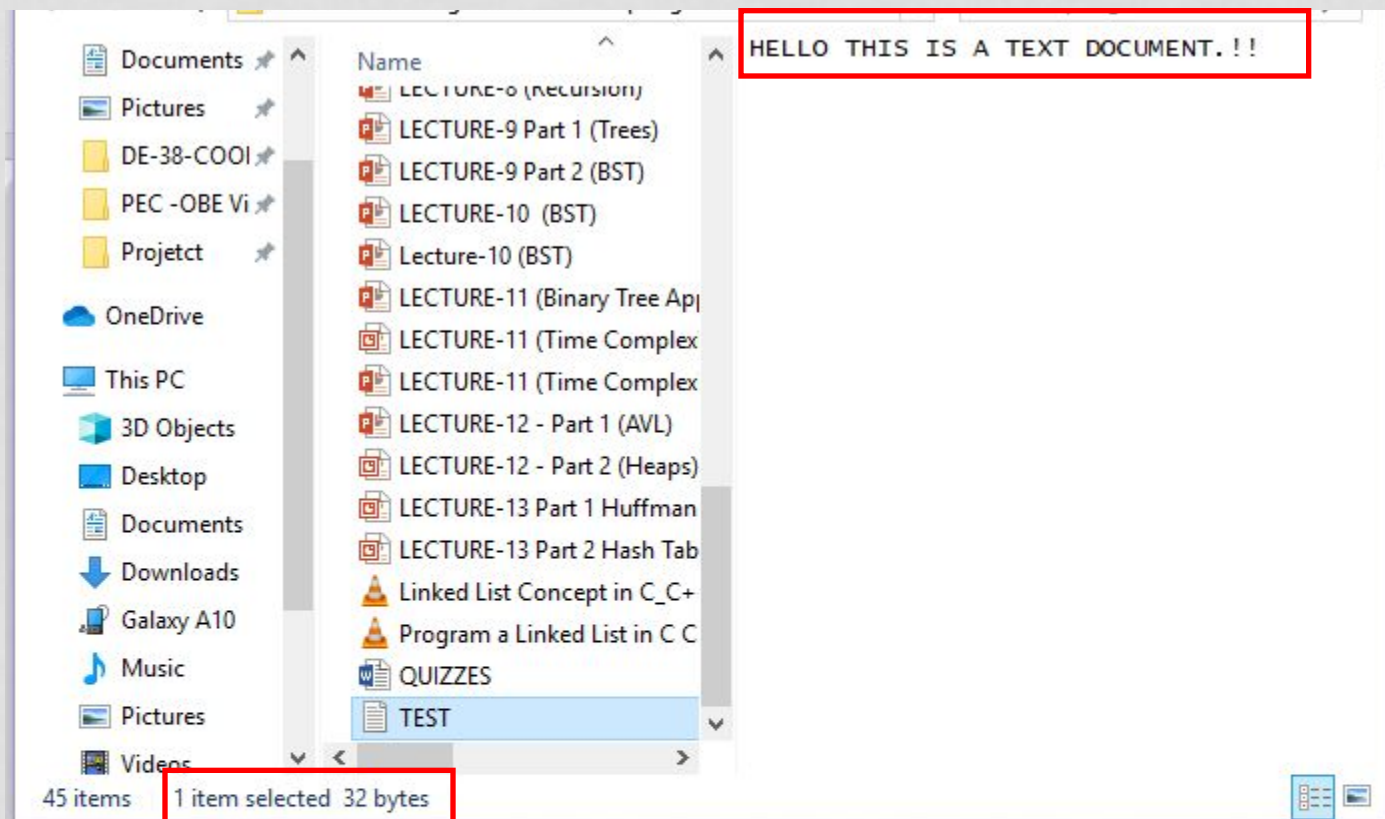
ASCII CODE

- How much size does following notepad file takes



ASCII CODE

- What if we modify the text file and add two more characters



ASCII CODE

- It's an 8-bit character code where every single bit represents a unique character.
- Using ASCII, 256 different characters are encoded which includes
 - Control Characters (ESC, SPACE BAR, ENTER . . .)
 - Special characters (~, % , °, ^, >, < . . .)
 - Punctuation marks (!, ., ?, &, “, :, ;, . . .)
 - Letters and Numbers (A, d, 5, h, H,)

$$2^8=256$$

HOW CAN IT BE IMPROVED

- It's an 8-bit character code where every single bit represents a unique character.
- Using ASCII, 256 different characters are encoded which includes

Ch	Occ	Ch	Occ	Ch	Occ	Ch	Occ	Ch	Occ	Ch	Occ
T	1	i	2	space	4	t	2	M	1	.	1
h	1	s	5	a	2	e	3	g	1		

- Letters and Numbers (A, d, 5, h, H,)

$$2^8=256$$

TREE APPLICATION

HUFFMAN CODING

- Proposed by Dr. David A. Huffman
 - A Method for the Construction of *Minimum Redundancy Codes*
 - Written in 1952
- Applicable to many forms of data
 - Our example: text files

THE BASIC ALGORITHM

- Huffman coding is a form of **statistical** coding
- Not all characters occur with the same frequency!
- Yet conventionally all characters are allocated the same amount of space while coding

1 char = 1 byte, be it **e** or **X**

EXAMPLE ENGLISH LANGUAGE

- It has 26 characters
- You require 2^5 bits per character to represent 32 unique combinations
- But each character is represented by 8-bits (ASCII)
- Lots of redundancy
- Not all characters occur with the same frequency!
- e is the most frequent character in English language

THE BASIC ALGORITHM

- Any savings in tailoring codes to frequency of character?
- Code word lengths are no longer fixed like **ASCII** or **Unicode**
- Varying length Codes
 - shorter for the more frequently used characters

THE (REAL) BASIC ALGORITHM

1. Scan text to be compressed and tally occurrence of all characters.
2. Sort or prioritize characters based on number of occurrences in text.
3. Build Huffman code tree based on the prioritized list.
4. Perform a traversal of tree to determine all code words.
5. Scan text again and create new file using the Huffman codes

EXAMPLE: BUILDING A TREE

SCAN THE ORIGINAL TEXT

- Consider the following short text

- Eerie eyes seen near lake.

26 byte

- Count up the occurrences of all characters in the text

BUILDING A TREE

SCAN THE ORIGINAL TEXT

← Eerie eyes seen near lake.

- What characters are present?

E e r i space
y s n a l k .

BUILDING A TREE

SCAN THE ORIGINAL TEXT

Eerie eyes seen near lake.

- What is the frequency of each character in the text?

Char	Freq.	Char	Freq.	Char	Freq.
E	1	y	1	k	1
e	8	s	1	.	1
r	2	n	2		
i	1	a	2		
space	4	1	1		

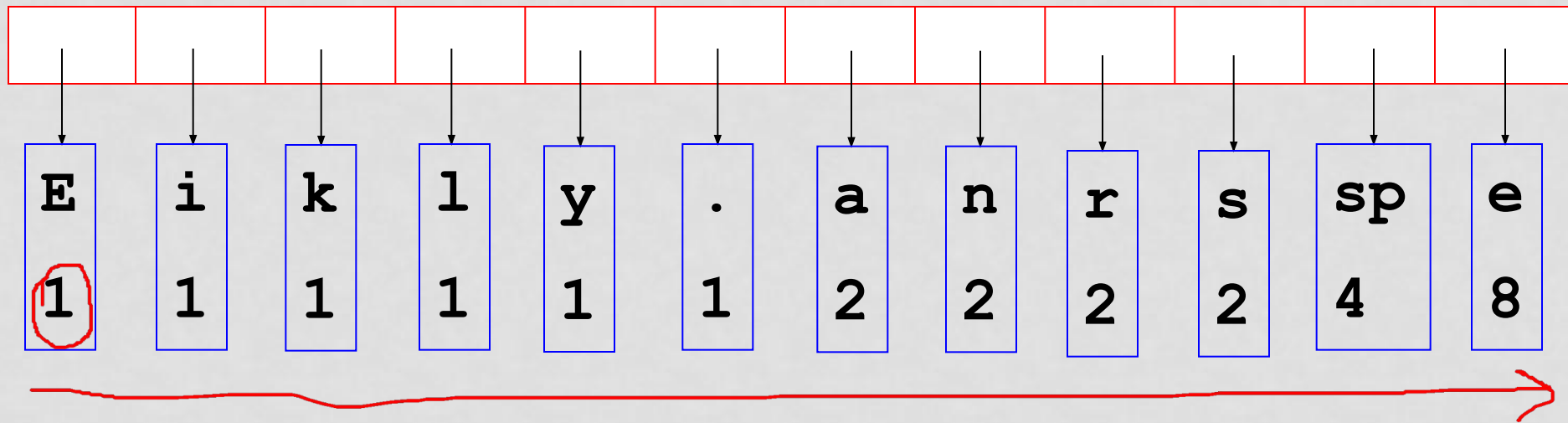
BUILDING A TREE

PRIORITIZE CHARACTERS

- Create binary tree nodes with character and frequency of that character
 - Place nodes in a **priority queue**
 - The lower the occurrence, the higher the priority in the queue
-

BUILDING A TREE

- The queue after inserting all nodes

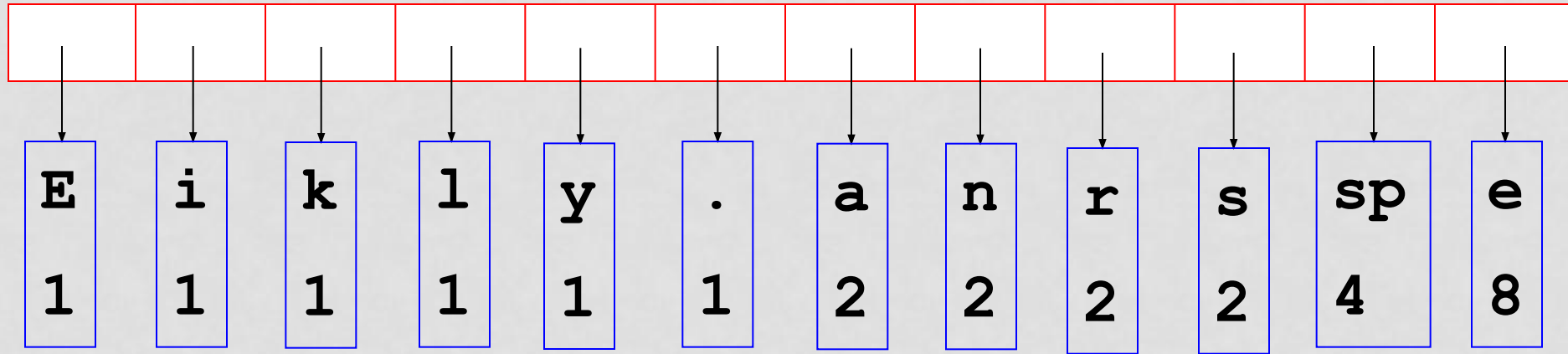


- Null Pointers are not shown

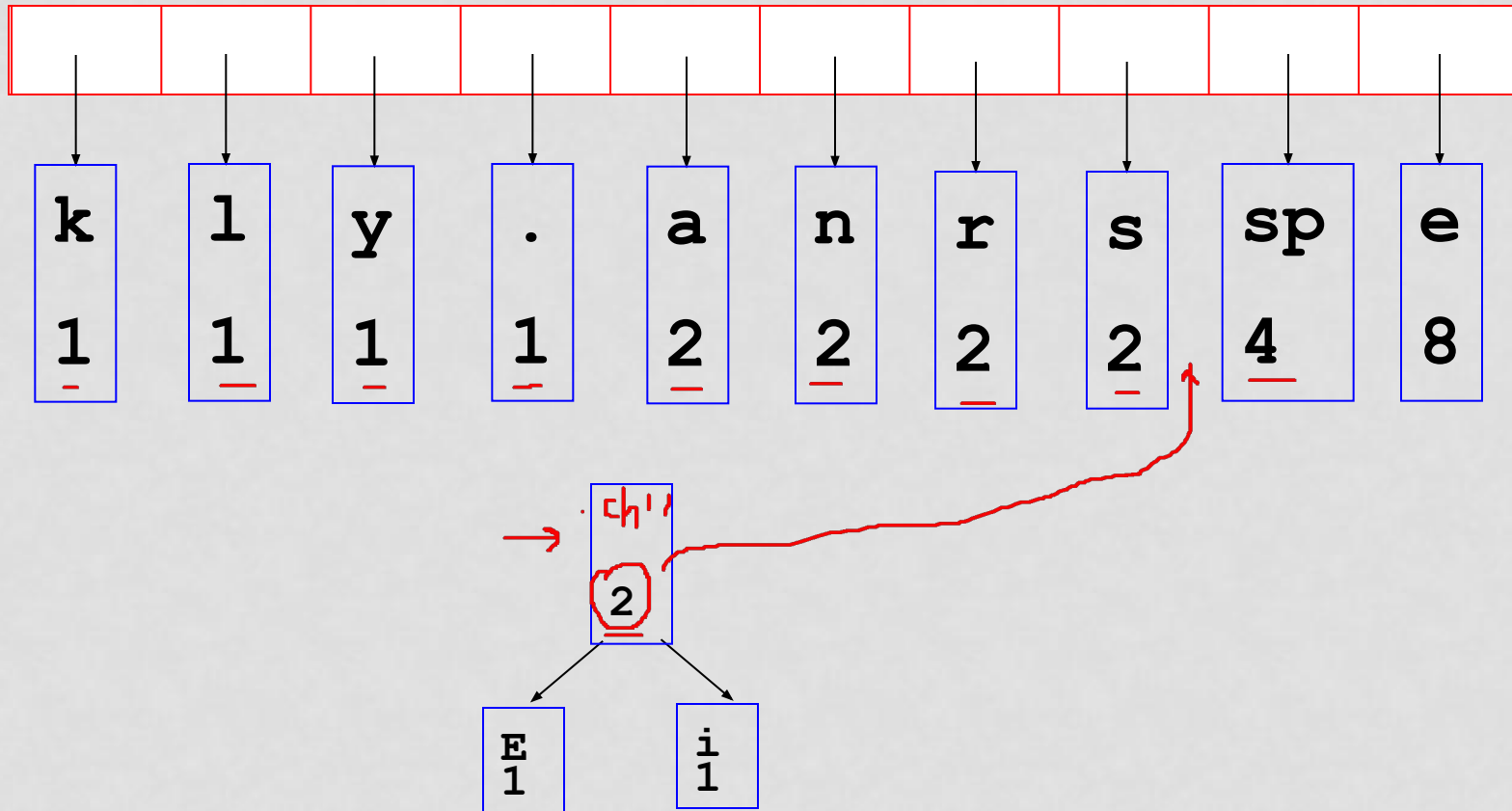
BUILDING A TREE

- While priority queue contains two or more nodes
 - Create new node
 - Dequeue node and make it left subtree
 - Dequeue next node and make it right subtree
 - Frequency of new node equals sum of frequency of left and right children
 - Enqueue new node back into queue

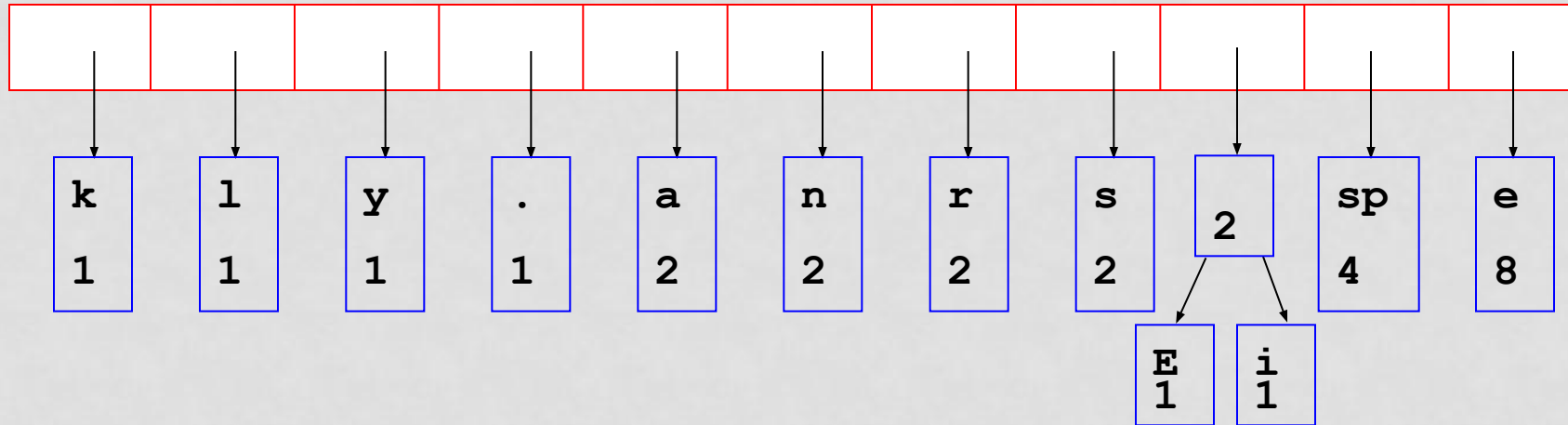
BUILDING A TREE



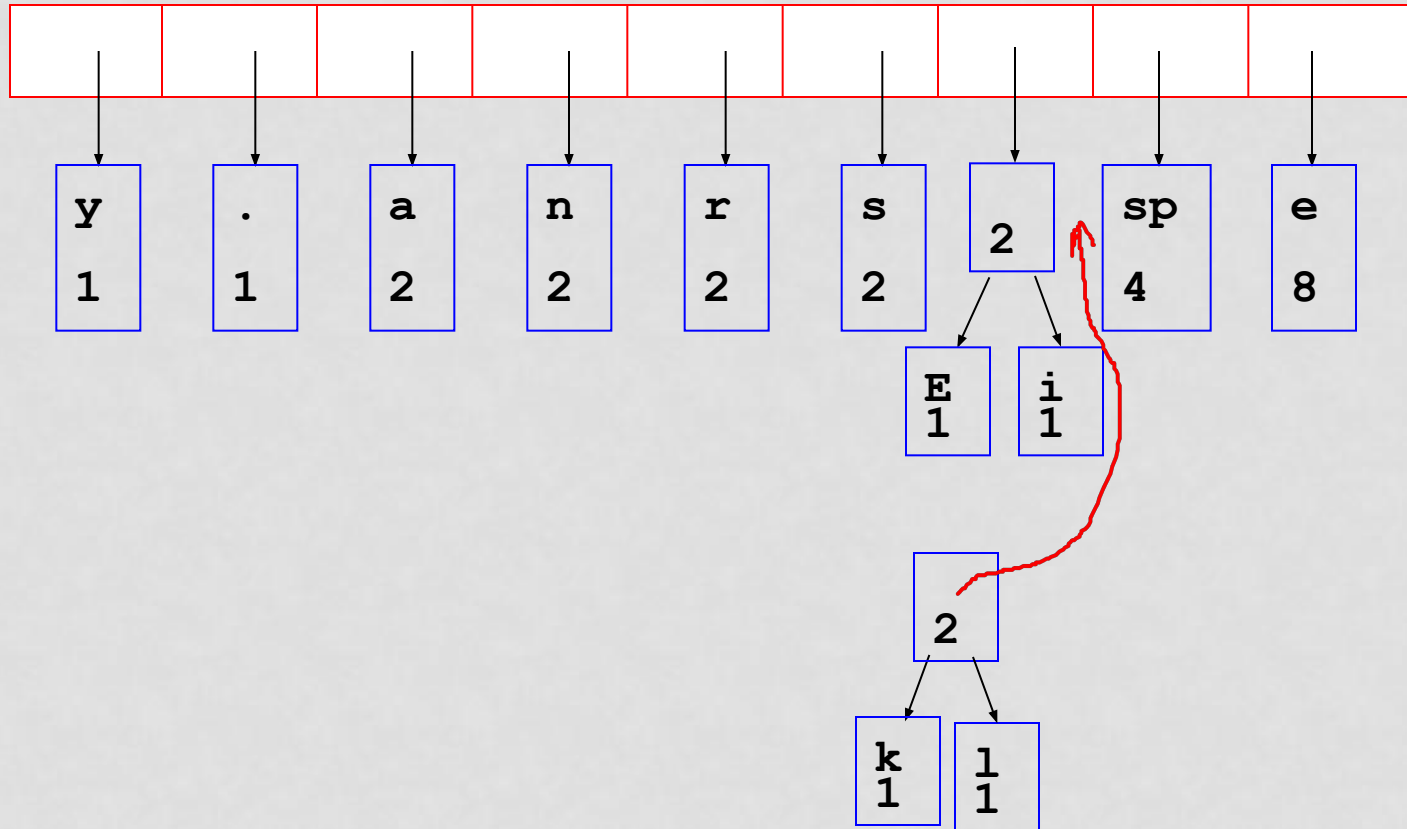
BUILDING A TREE



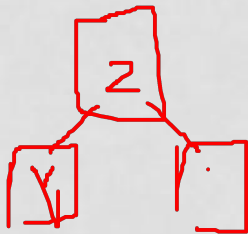
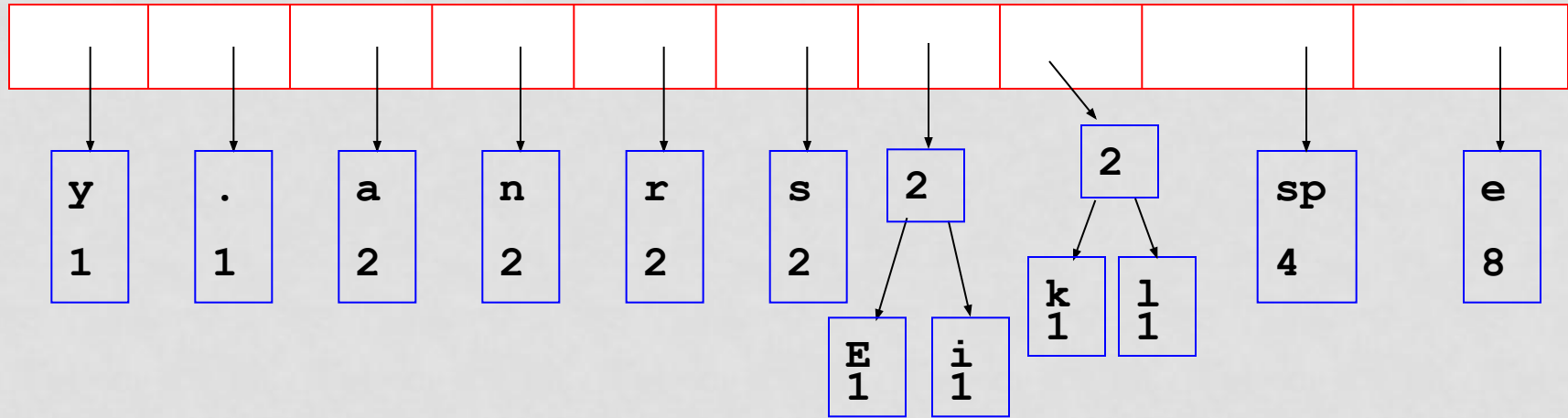
BUILDING A TREE



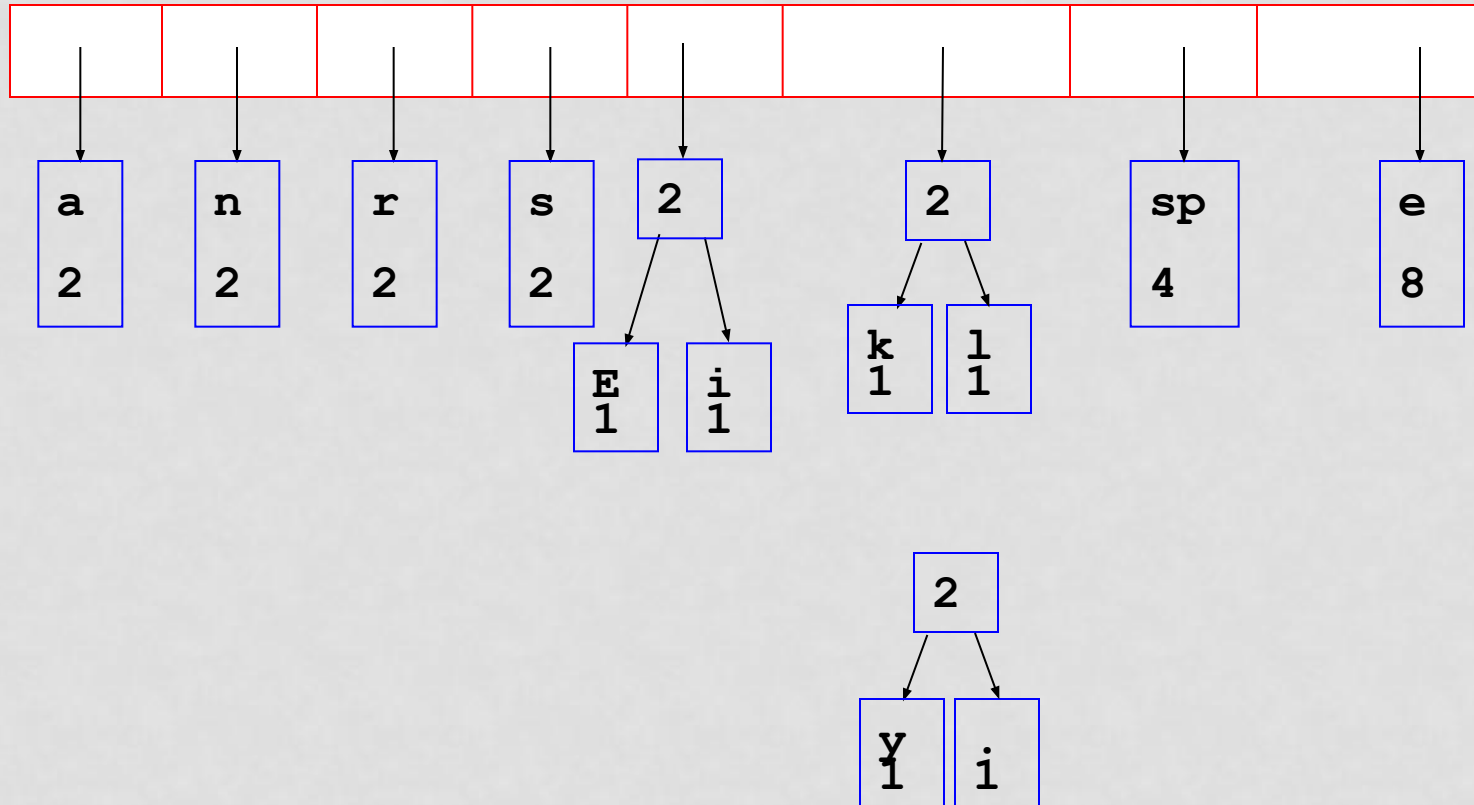
BUILDING A TREE



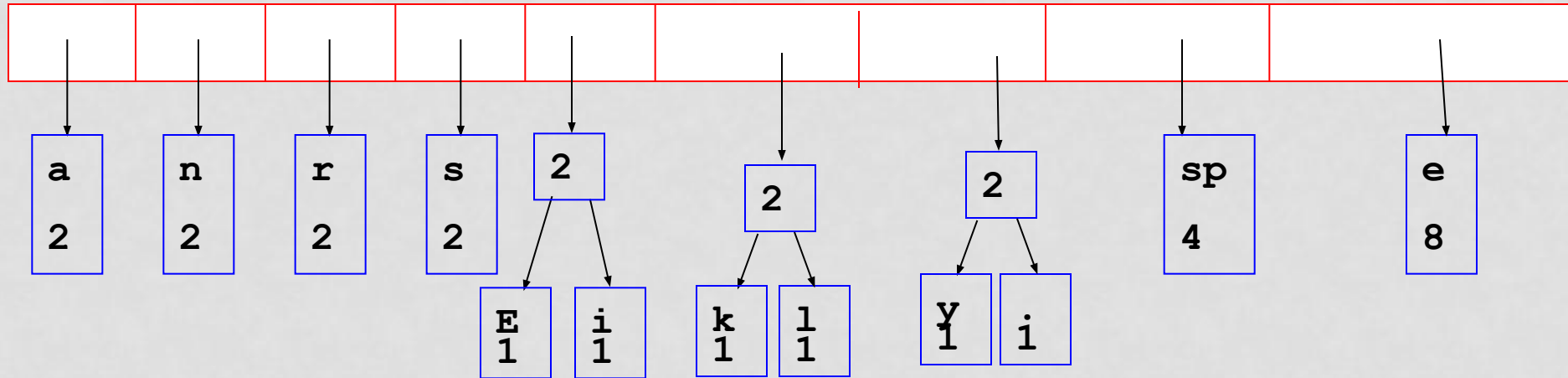
BUILDING A TREE



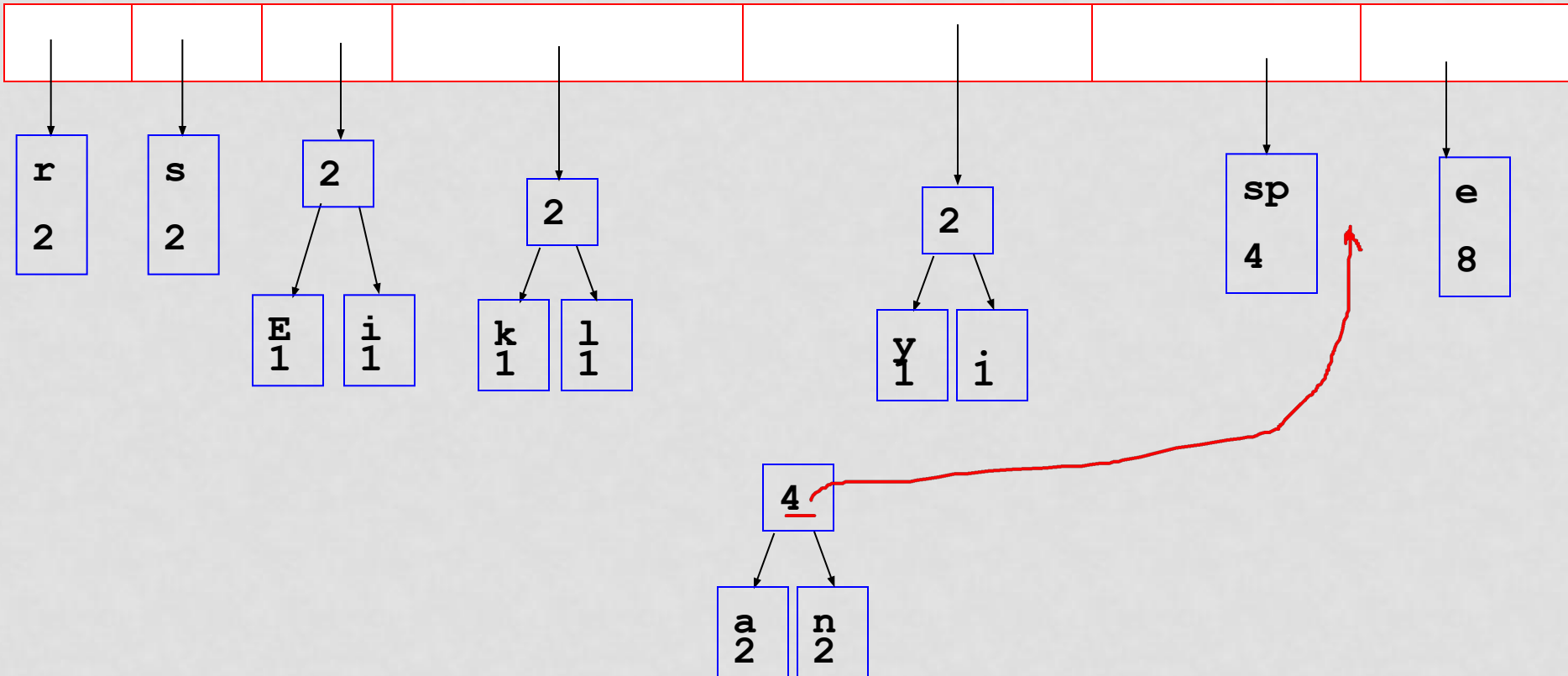
BUILDING A TREE



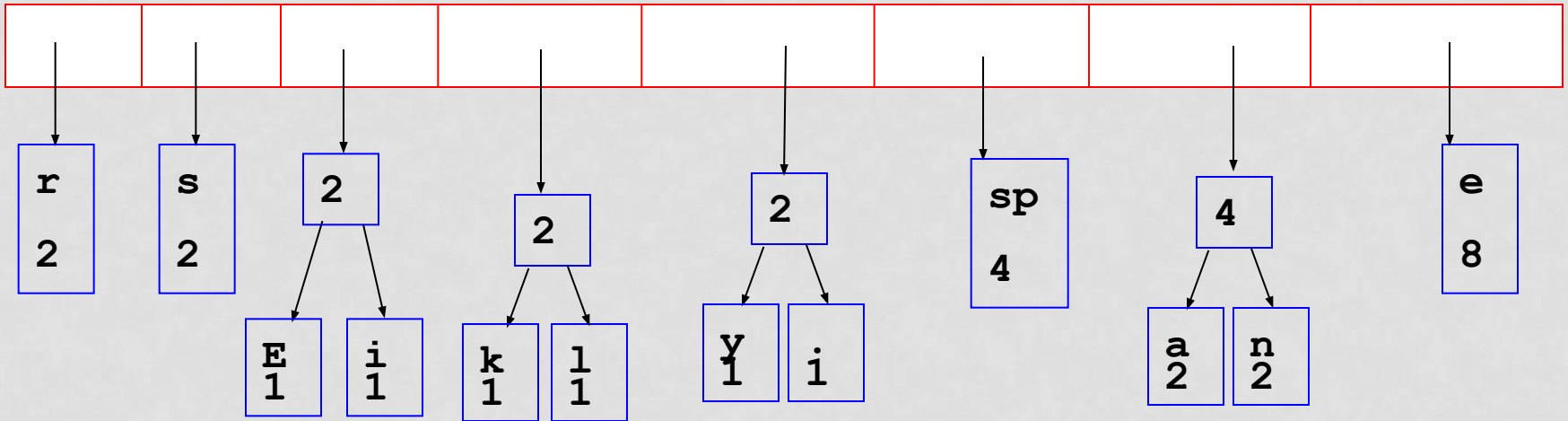
BUILDING A TREE



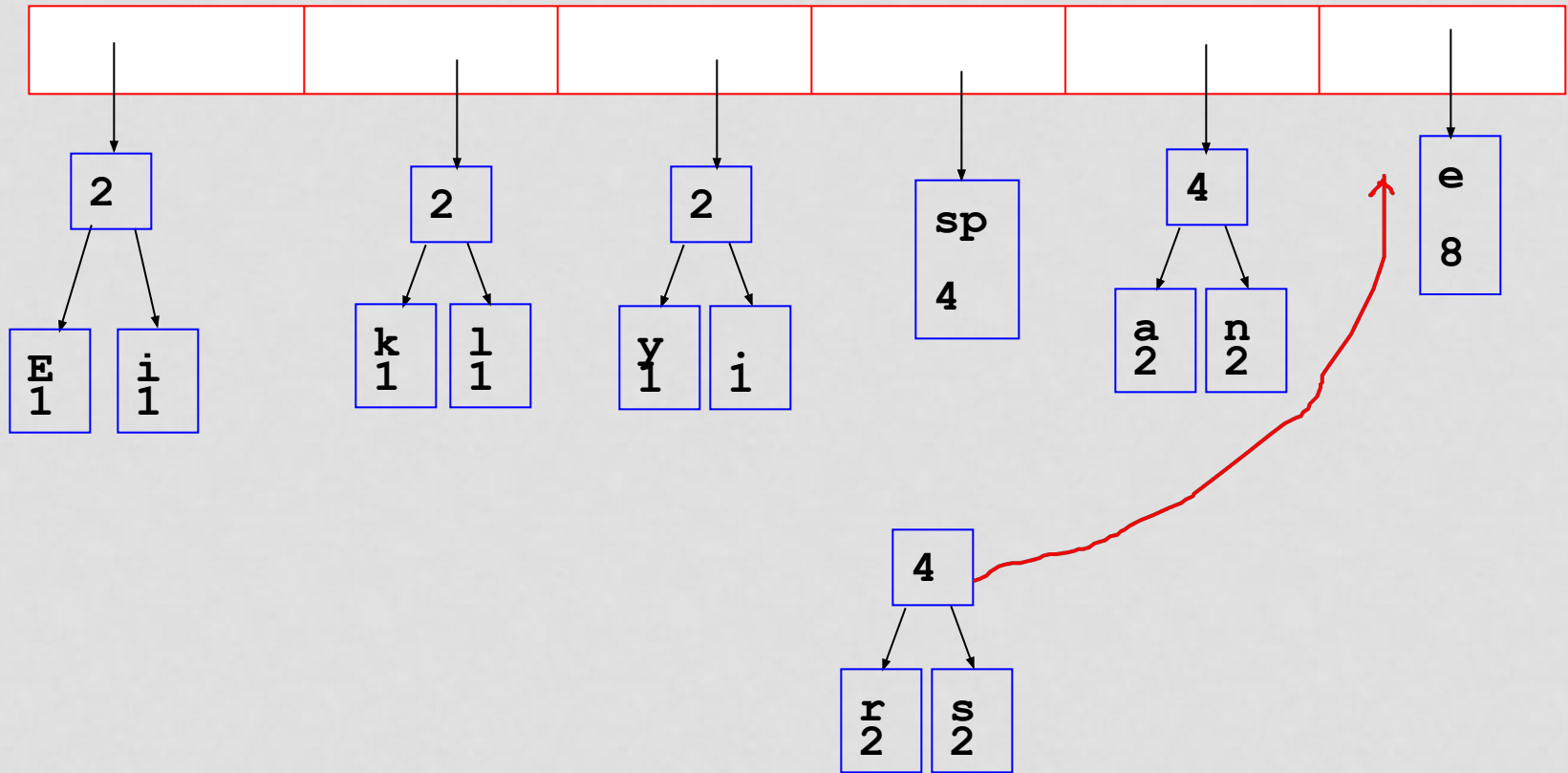
BUILDING A TREE



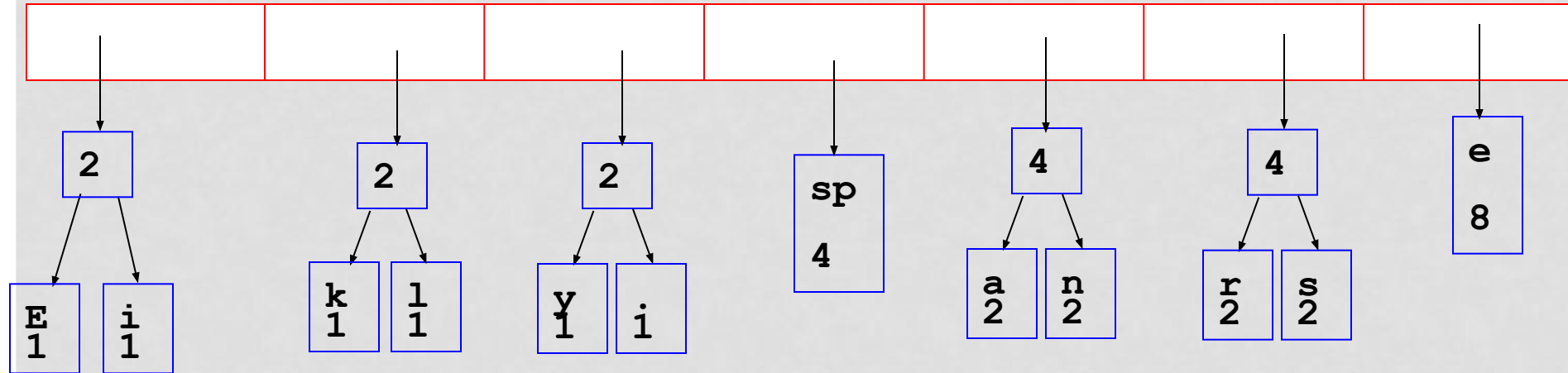
BUILDING A TREE



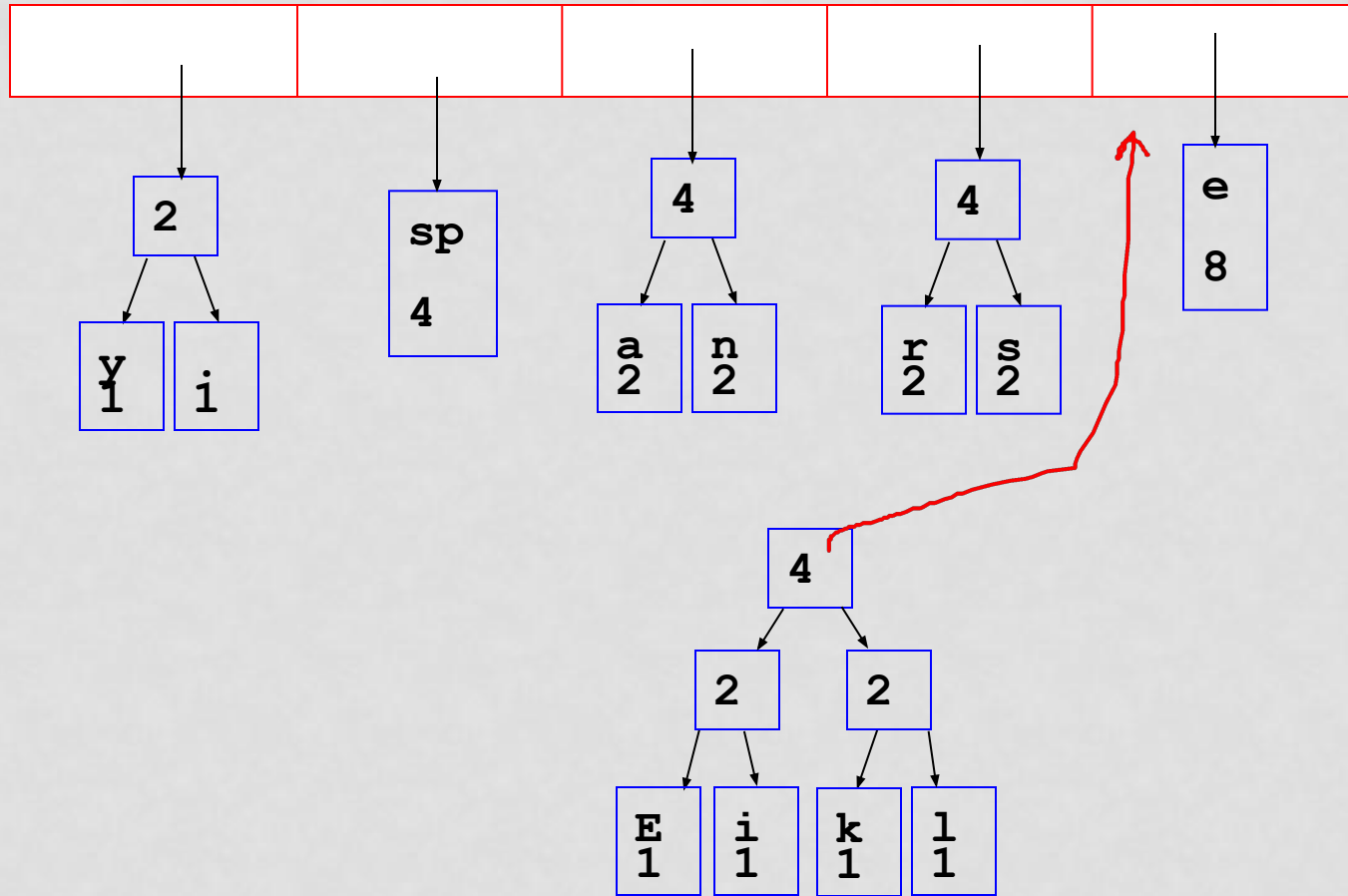
BUILDING A TREE



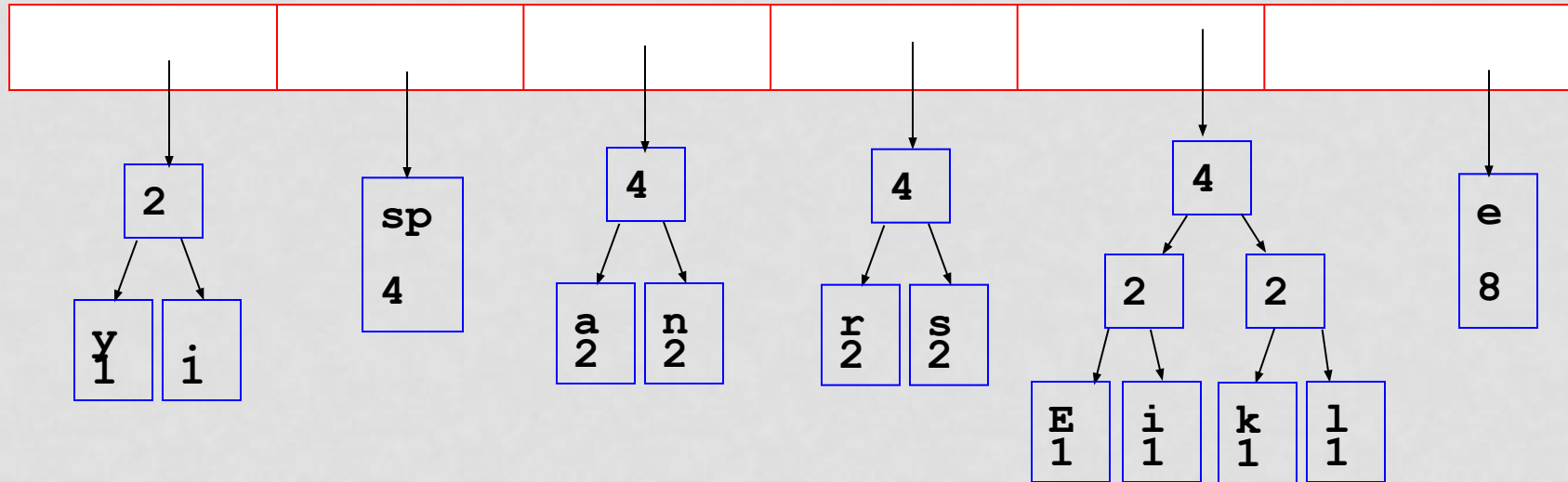
BUILDING A TREE



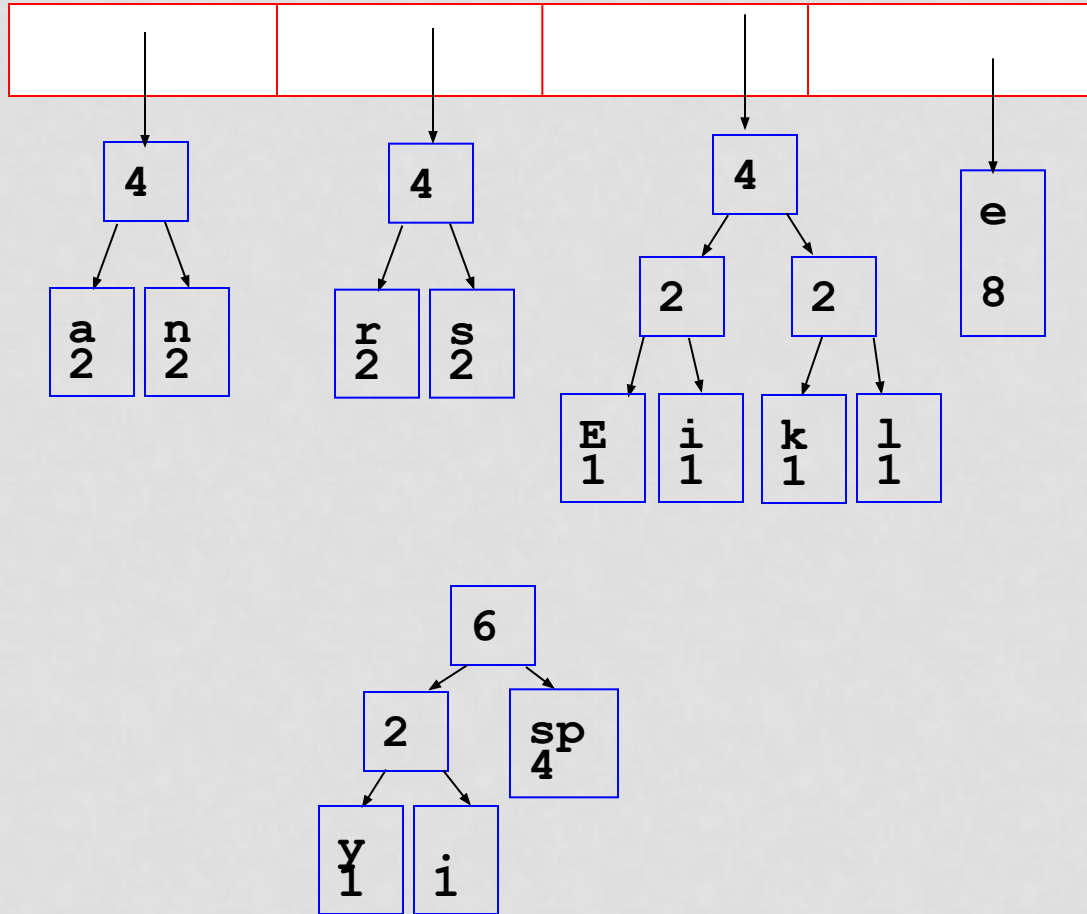
BUILDING A TREE



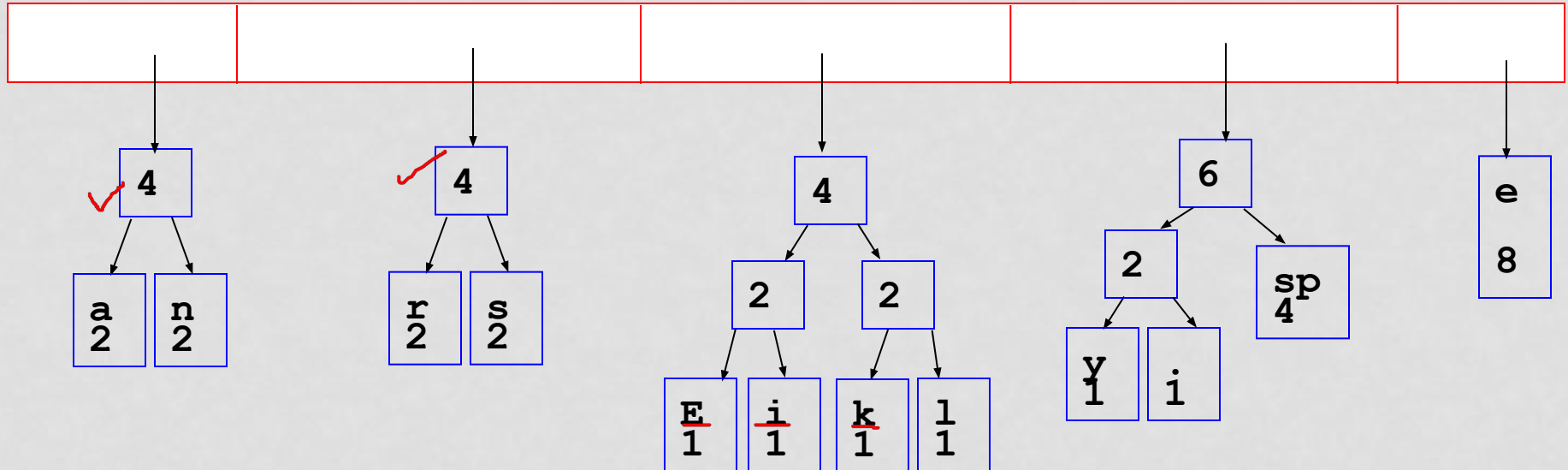
BUILDING A TREE



BUILDING A TREE

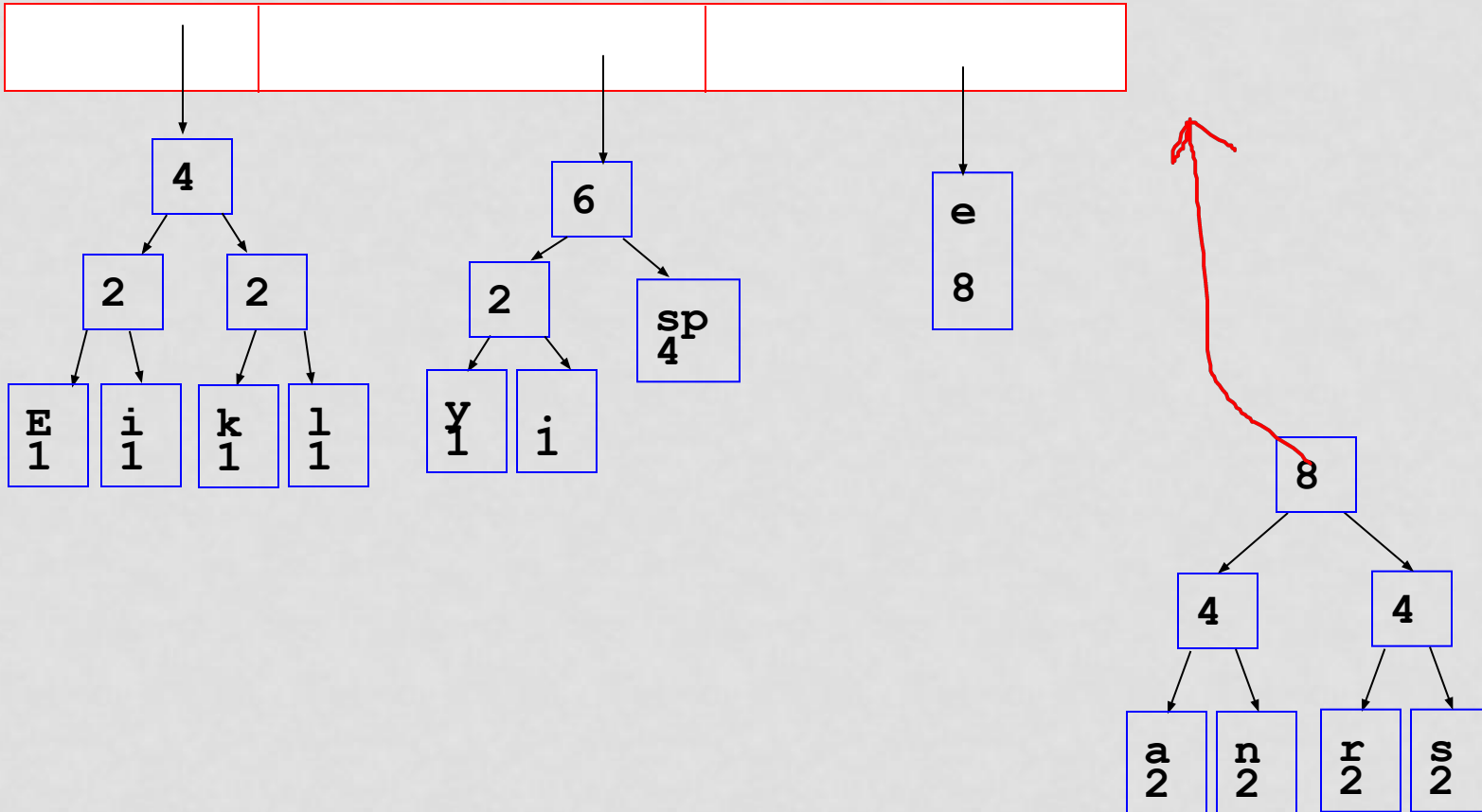


BUILDING A TREE

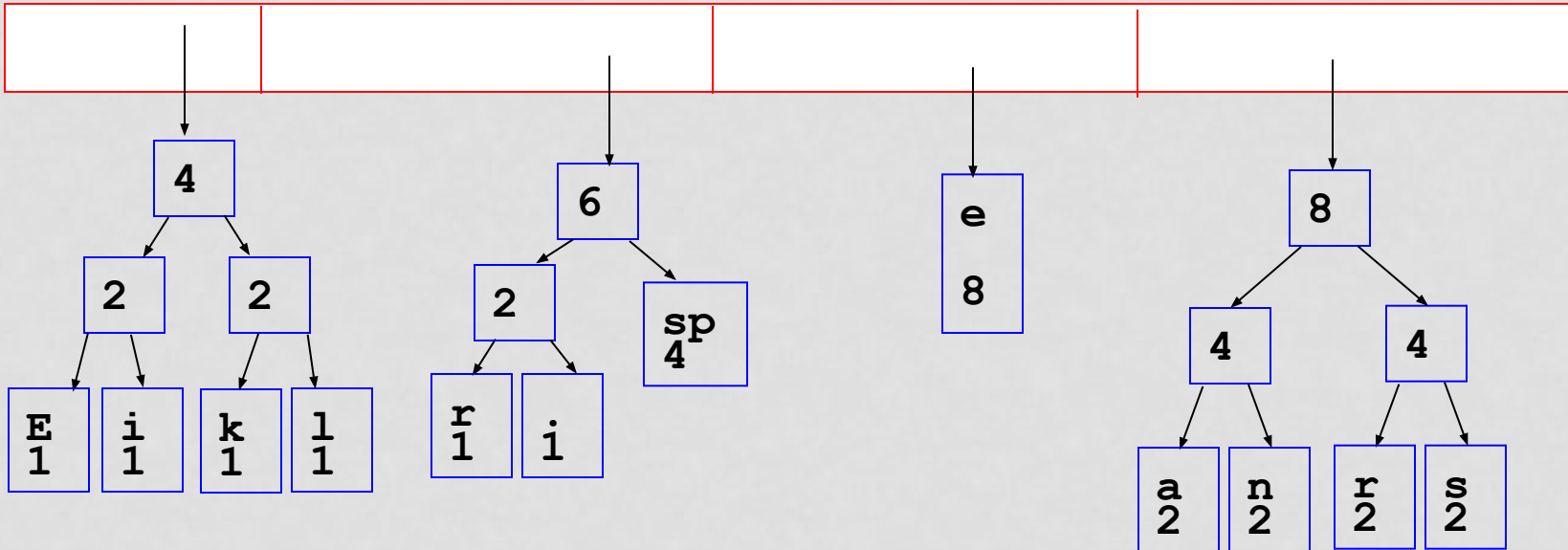


What is happening to the characters with a low number of occurrences?

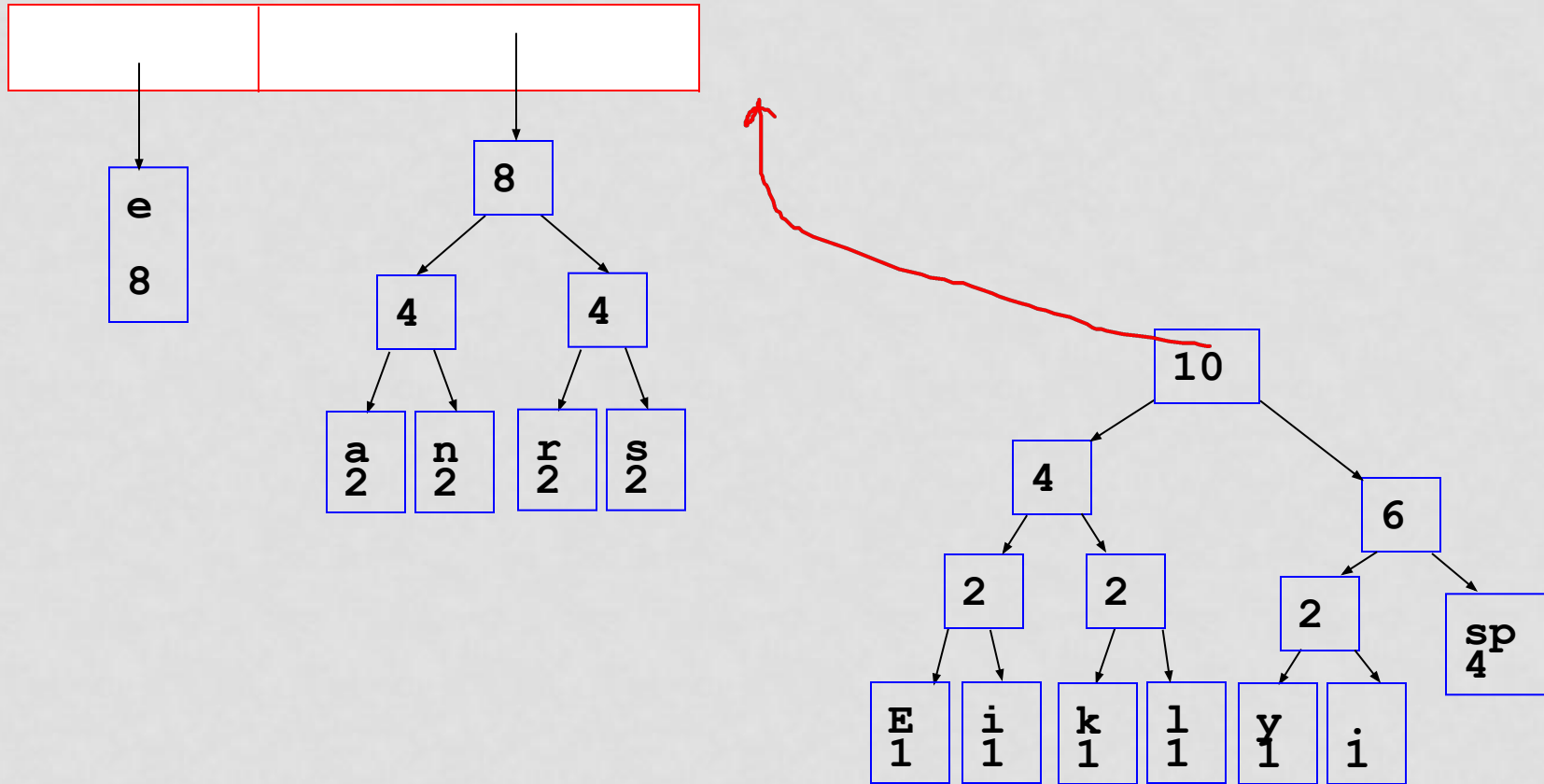
BUILDING A TREE



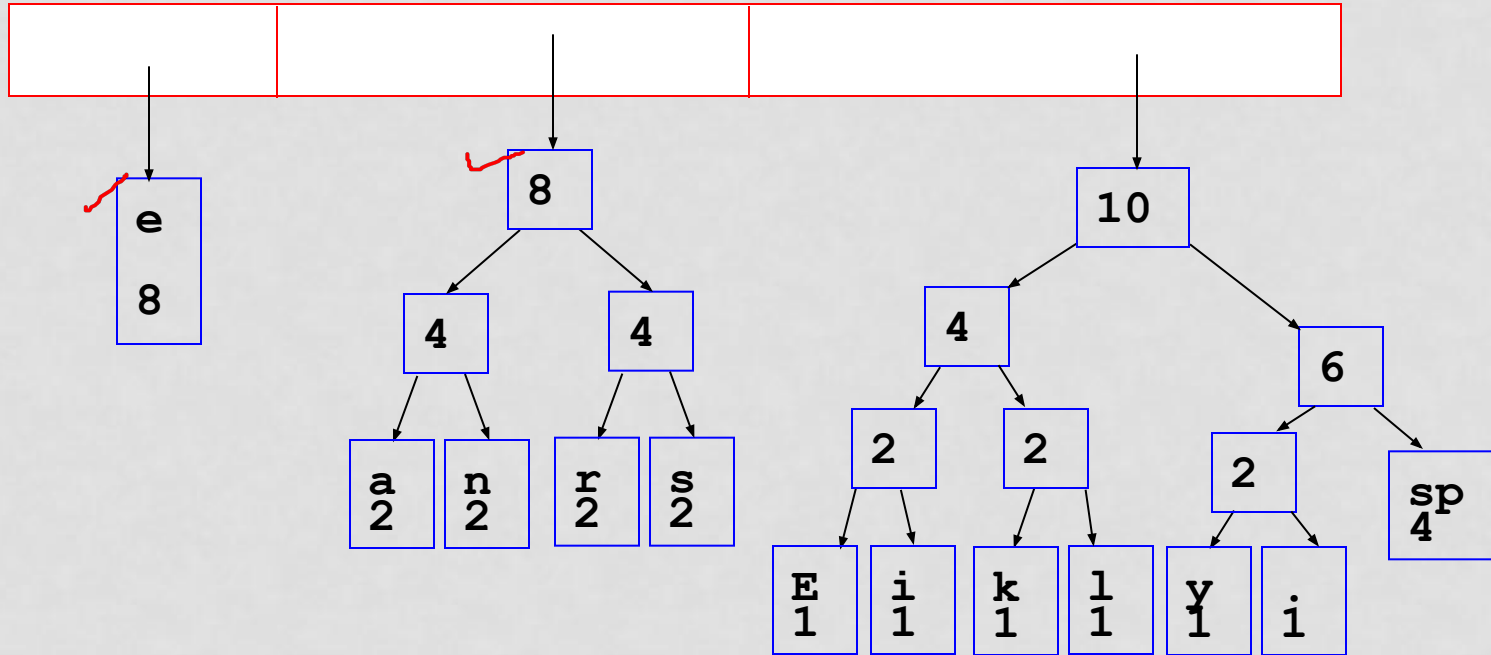
BUILDING A TREE



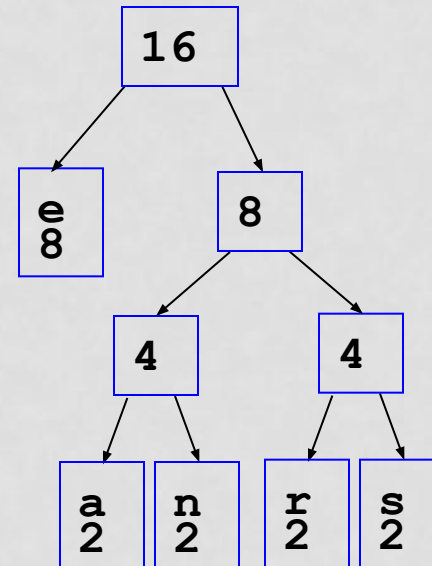
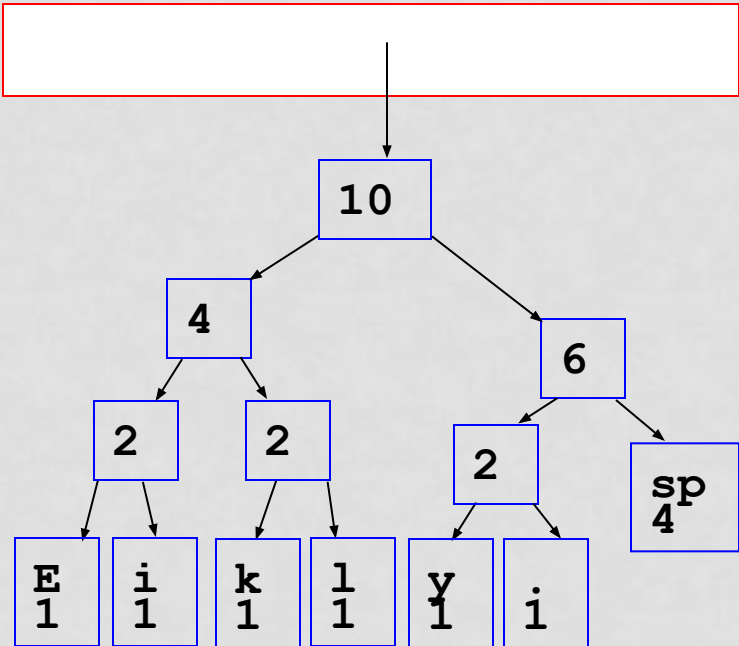
BUILDING A TREE



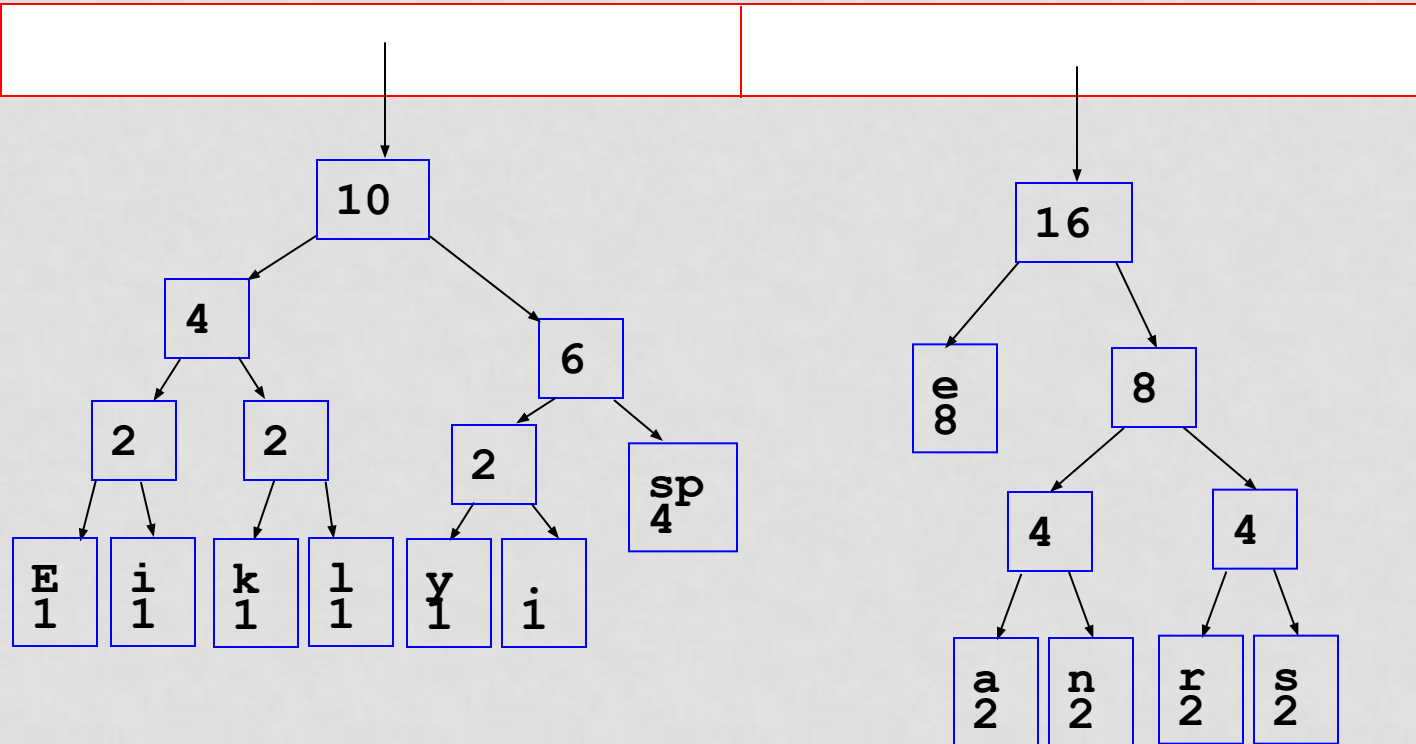
BUILDING A TREE



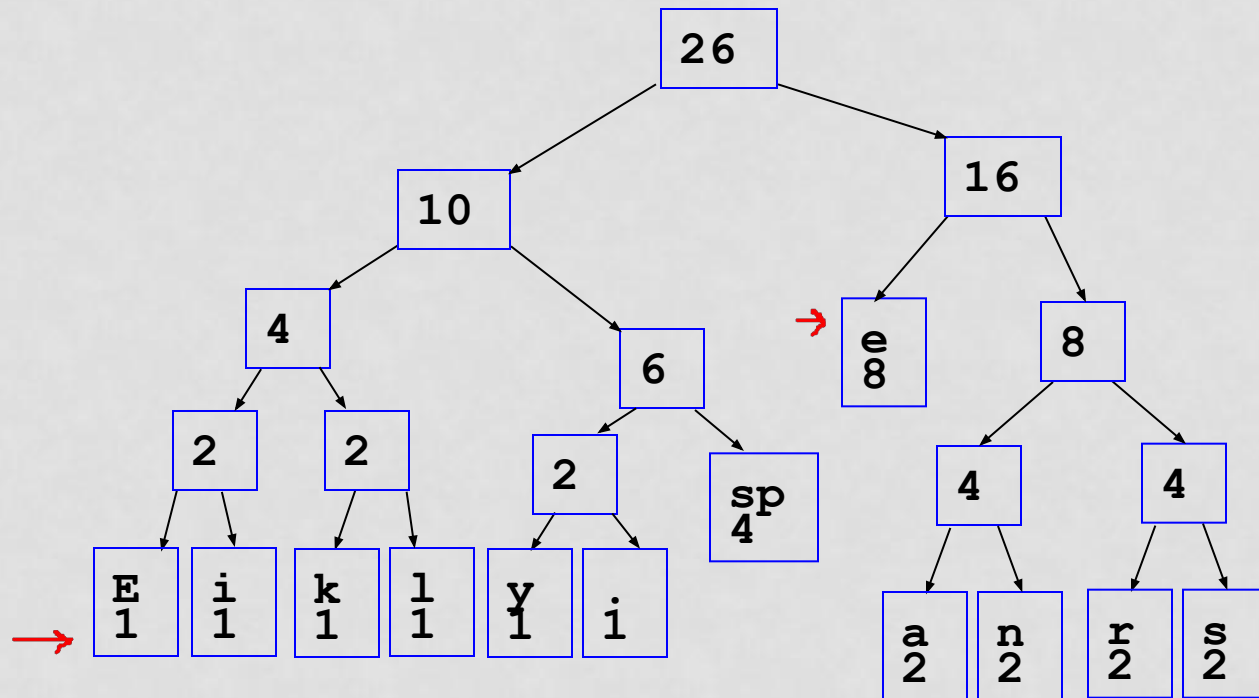
BUILDING A TREE



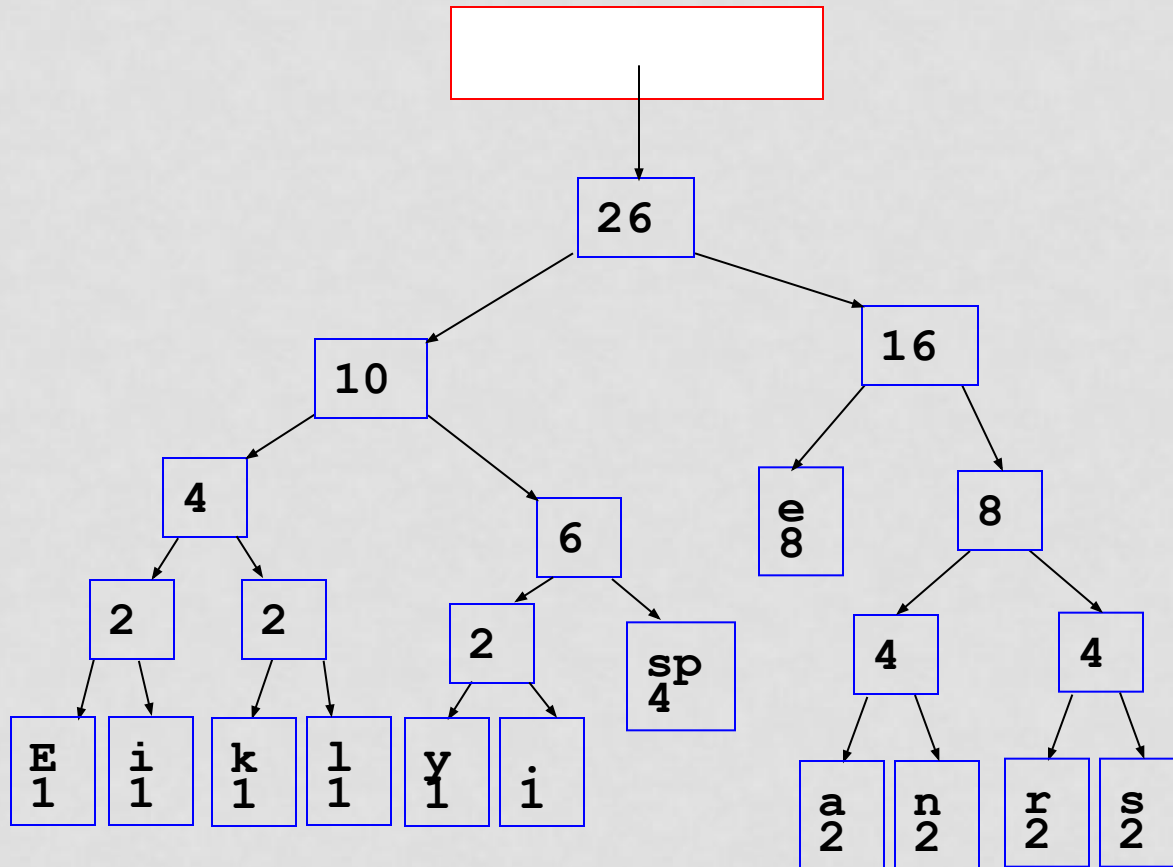
BUILDING A TREE



BUILDING A TREE



BUILDING A TREE



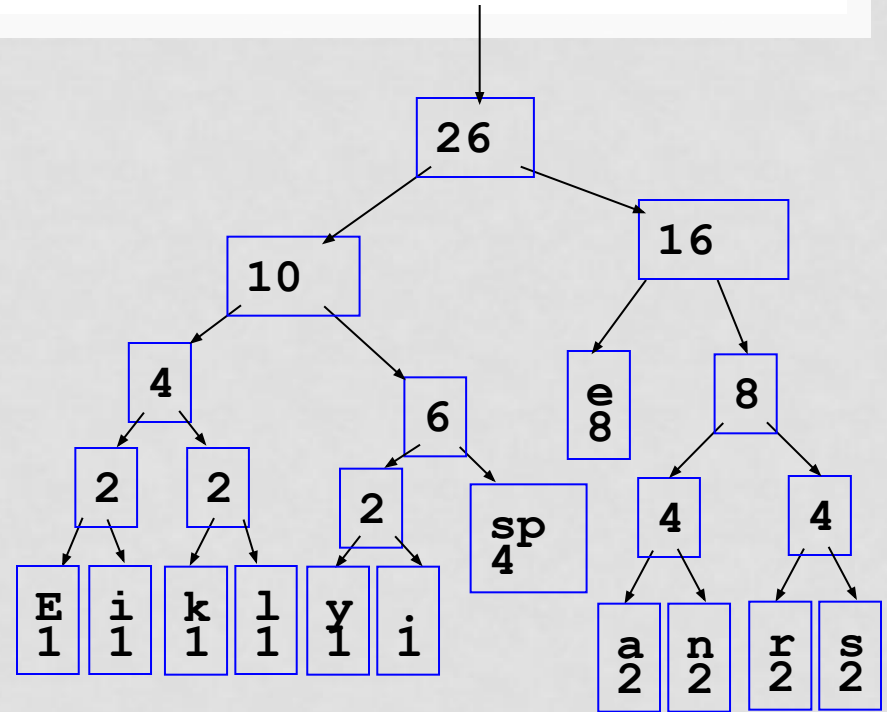
- After enqueueing this node there is only one node left in priority queue.

BUILDING A TREE

Dequeue the single node left in the queue.

This tree contains the new code words for each character.

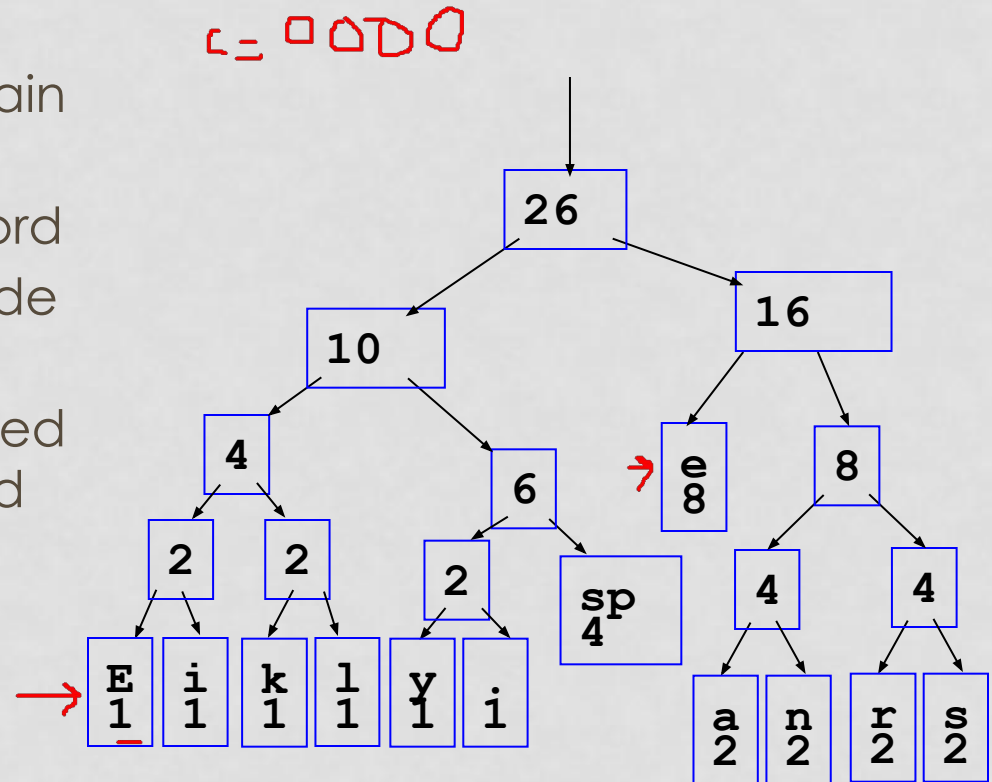
Frequency of root node should equal number of characters in text.



ENCODING THE FILE

TRAVERSE TREE FOR CODES

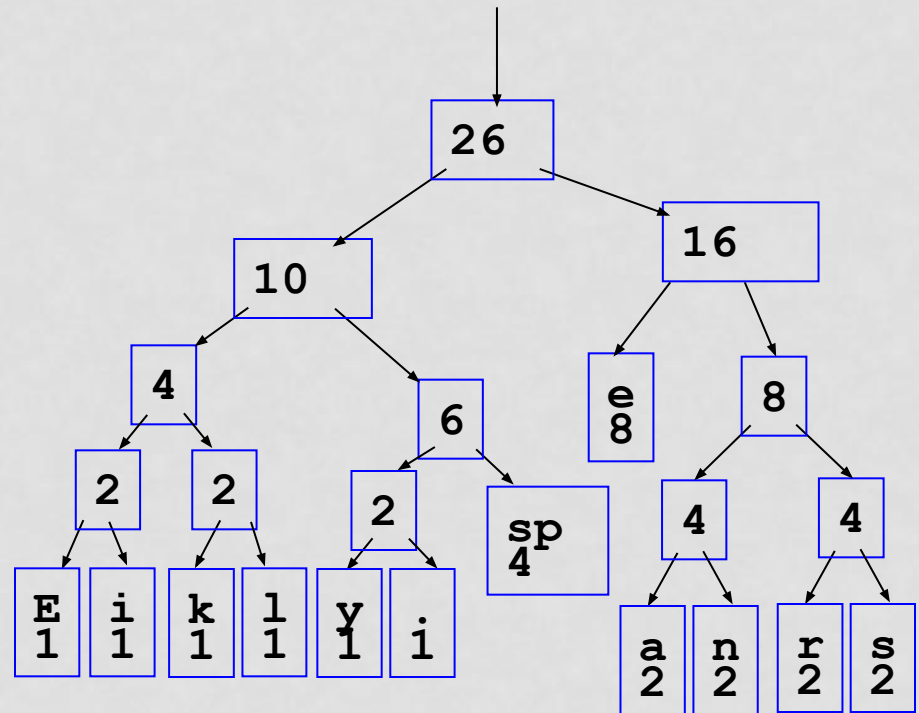
- Step down the tree to obtain new code words
- **left, append a 0** to code word
- **Right, append a 1** to code word
- code word is only completed when a leaf node is reached



ENCODING THE FILE

TRAVERSE TREE FOR CODES

Char	Code
E	0000
i	0001
k	0010
l	0011
y	0100
.	0101
space	011
e	10
a	1100
n	1101
r	1110
s	1111



ENCODING THE FILE

- Rescan text and encode file using new code words
Eerie eyes seen near lake.

00001011100001100111001
00100111111101011010111
10110110011100110011110
00010100101

→ Left

- Why is there no need for a separator character?

Char	Code
E	0000
i	0001
k	0010
l	0011
y	0100
.	0101
space	011
e	10
a	1100
n	1101
r	1110
s	1111

ENCODING THE FILE RESULTS

- Have we made things any better?
- Huffman coding used 73 bits to encode the text
- ASCII would take $8 * 26 = 208$ bits

```
00001011100001100111001
00100111111101011010111
10110110011100110011110
00010100101
```

DECODING THE FILE

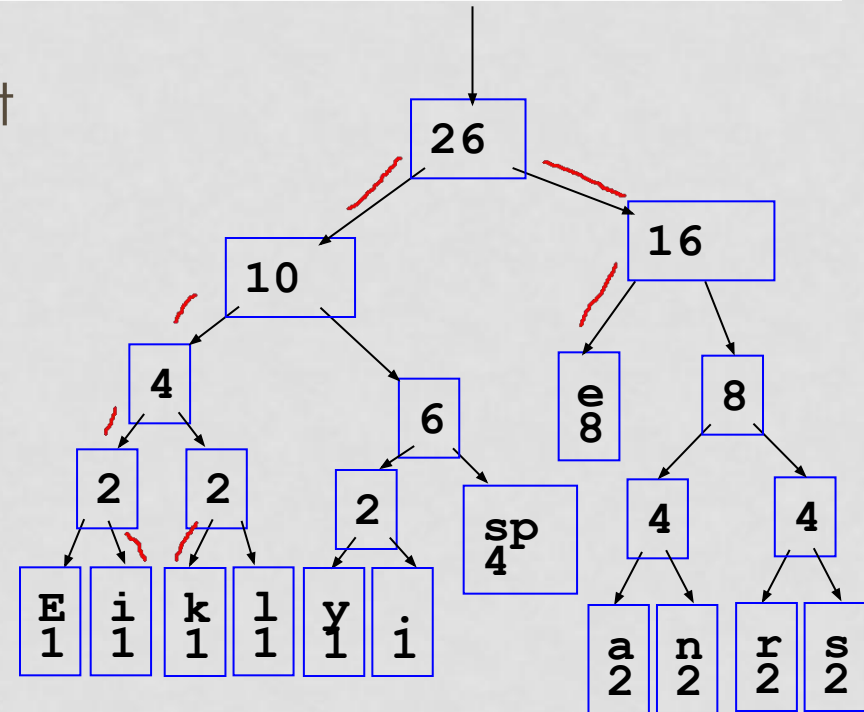
- How does receiver know what the codes are?
- Tree predetermined
 - based on statistical analysis of text files or file types
- Data transmission is bit based (not byte based)

DECODING THE FILE

- Once receiver has tree it scans incoming bit stream
- 0 $\Rightarrow$ go left
- 1 $\Rightarrow$ go right

1010001001111000111111
11011100001010

ack



REFERENCES

- Halsall, F., *Data Communications, Computer Networks and Open Systems Fourth Edition*, Addison-Wesley, (1996)
- Useful web sites
 - <http://www.rasip.etf.hr/research/compress/algorithms/fund/huffman/index.html>
 - http://www.eee.bham.ac.uk/WoolleySI/All7/huff_1.htm
 - <http://www.compressconsult.com/huffman/>